FACULTY OF INFORMATION TECHNOLOGY
VNUHCM – University of Science

Advanced DevOps — Course Project

Deploying a Microservice Application on Kubernetes with Ansible and Helm

Two-node cluster with Vagrant · kubeadm via Ansible · Online Boutique via Ansible Helm

Field	Value	Field	Value
Duration	2 weeks	**Total marks**	100 (+10 bonus)
Work mode	Individual or group with max 3 students	**Weight**	10% of course grade
Deliverable	Git repository + report + demo	**Demo**	Live, in the lab session

1. What You Are Building

You will stand up a two-node (or three-node) Kubernetes cluster on your own machine and deploy a real eleven-component microservice application onto it — the Google "Online Boutique" web store. Every step after creating the virtual machines is automated with Ansible. You will not type a single kubectl command to deploy the application.

The cluster looks like this:

```
Cluster  (VirtualBox host) , each PC/Laptop can run 1 VM
│
├── master   192.168.56.10   2 vCPU   2048 MB   control plane
│            kube-apiserver, etcd, scheduler, controller-manager
│
└── node1    192.168.56.11   2 vCPU   4096 MB   worker
             │
             └── namespace: microservice
                 ├── frontend ............ Service type NodePort
                 │                                ↑ your browser connects here
                 ├── adservice            checkoutservice
                 ├── cartservice          currencyservice
                 ├── emailservice         paymentservice
                 ├── productcatalogservice shippingservice
                 ├── recommendationservice
                 └── redis-cart .......... shopping cart storage
```

Ten of the eleven components are internal ClusterIP Services that talk to each other over gRPC. Only the frontend is reachable from outside the cluster.

The point of this project. Anyone can follow a tutorial that runs kubectl apply. What this project assesses is whether you can make an upstream Helm chart — written by somebody else, for a different platform, with a bug in it — deploy correctly on infrastructure you built yourself, and prove that it works.

2. Learning Outcomes

On completing this project you should be able to:

- Provision a multi-node Kubernetes cluster with kubeadm, driven entirely by Ansible roles and a static inventory.
- Explain why a cluster built on Vagrant needs explicit network configuration, and configure both the CNI plugin and the kubelet to use the correct interface.
- Deploy a multi-release Helm application from Ansible using the kubernetes.core collection.
- Adapt a third-party Helm chart to a new environment through values overrides, without forking or editing the upstream source.
- Diagnose a broken microservice deployment by reading Service names, environment variables and pod logs, rather than by guessing.
- Write automated assertions that prove a deployment works, instead of asserting it in prose.

3. Prerequisites

Software on the host

Tool	Version	Check with
VirtualBox	7.0 or newer	`VBoxManage --version`
Vagrant	2.3 or newer	`vagrant --version`
Ansible	2.14 or newer	`ansible --version`
Git	any recent	`git --version`

4. The Project

Basing on:

1. Create by yourself OR
2. https://github.com/geerlingguy/ansible-for-devops/tree/master/kubernetes

Why the Vagrantfile has no provisioner

The book examples run Ansible automatically from Vagrant. This project does not. You run the playbooks yourself against a static inventory, exactly as you would against real servers. Vagrant is only a convenient way to obtain two Linux machines.

5. Part 1 — Create the Virtual Machines

1. Install the Ansible roles and collections the project depends on. They land in ./roles and ./collections, which are git-ignored.

```
$ ansible-galaxy install -r requirements.yml
```

2. Create both VMs. This downloads the Ubuntu 22.04 base box the first time and takes a few minutes.

```
$ vagrant up
```

3. Confirm both machines are running, then confirm Ansible can reach them over the private network using the keys Vagrant generated.

```
$ vagrant status
$ ansible k8s -m ping
```

You should see a green SUCCESS from both master and node1. If you see a permission error, the key paths in inventory.ini only exist after `vagrant up` has completed.

6. Part 2 — Build the Kubernetes Cluster

One command builds the cluster: (ansible-for-devops/kubernetes/main.yml)

```
Example

$ ansible-playbook playbooks/cluster.yml
```

Expect eight to twelve minutes. While it runs, read what it is doing.

What the playbook does, in order

Stage	Why it is necessary
Disable swap, in fstab too	The kubelet refuses to start while swap is enabled. The base box ships with it on.
Load the overlay module, enable net.ipv4.ip_forward	containerd needs overlayfs; without IP forwarding, pod traffic between nodes is dropped by the kernel.
Download and patch the Flannel manifest	See below. This is the single most important step for a Vagrant cluster.

Stage	Why it is necessary
Pin the kubelet node IP	Also below. Same root cause, different symptom.
`geerlingguy.containerd`	Installs the container runtime with the systemd cgroup driver, which is what the kubelet expects.
`geerlingguy.kubernetes`	Installs kubeadm/kubelet/kubectl, runs kubeadm init on the control plane, applies the CNI, then joins node1 with a generated token.
`geerlingguy.helm`	Installs the Helm 3 binary on the control plane only.
Wait for Ready	The play does not finish until both nodes and all kube-system pods report Ready.

The two Vagrant networking traps

Every VirtualBox VM created by Vagrant has two network interfaces:

- `enp0s3` — the Bridge adapter.
- `enp0s8` — the private network, `192.168.56.x`. This is the only interface on which the VMs can actually reach each other.

7. Part 3 — Deploy the Online Boutique

One command deploys the application:

Example: https://github.com/geerlingguy/ansible-for-devops/blob/master/kubernetes/examples/helm.yml

```
Example

$ ansible-playbook examples/helm.yml
```

Where the charts come from

The Helm charts are not ours. The playbook clones them at run time from:

```
https://github.com/iankisali/microservice-deployment
```

That repository contains two generic charts — chart/microservice and chart/redis — plus one values file per service. A single parameterised chart deployed eleven times with different values is a common and sensible pattern; read chart/microservice/templates/ and you will see how little there is to it.

The override strategy

The upstream repository was written to deploy onto Amazon EKS. Three things about it are wrong for our cluster. We fix none of them by editing the clone. Instead, vars/microservices.yml layers overrides on top, and Helm resolves the precedence:

```
kubernetes.core.helm:
  values_files:                     # upstream file, unchanged
    - "{{ src }}/{{ item.values_file }}"
  values: "{{ common | combine(item.overrides | default({})) }}"
                                    # our overrides, applied last and winning
```

This matters beyond tidiness: the clone can be re-pulled at any time, and a diff of our repository shows precisely what we changed and why.

The three overrides

Override	Applies to	Reason
`appReplicas: 1`	every release	Upstream asks for two replicas of all ten services — over twenty pods. One 5 GB worker cannot schedule that. One replica each still exercises the full architecture.
`serviceType: NodePort`	frontend	Upstream asks for a LoadBalancer. On EKS a cloud controller provisions one; on bare-metal kubeadm nothing does, so the Service would sit in <pending> forever and the site would be unreachable.
`appName: checkoutservice`	checkoutservice	An upstream bug. See below.

The upstream bug

values/checkout-service-values.yaml opens with these two lines:

```
appImage: checkoutservice
appImage: gcr.io/google-samples/microservices-demo/checkoutservice
```

The first line clearly means appName. Because the key is duplicated, the second line wins and appName is never set at all — so the chart falls back to its default, which is the literal string "servicename". The Deployment and the Service are both created under that name. Meanwhile the frontend is configured with:

```
CHECKOUT_SERVICE_ADDR = checkoutservice:5050
```

which resolves to nothing. The storefront loads perfectly, you can browse products and fill a cart, and then checkout fails at the final step. This is a realistic failure: the symptom is nowhere near the cause.

Verified behaviour. Rendering the upstream chart with the upstream values really does produce a Deployment named "servicename".

8. Part 4 — Verify and Use the Application

The deployment playbook prints the URL when it finishes. It looks like:

```
http://192.168.56.11:31234/
```

The port is allocated from the NodePort range 30000–32767, so it differs on every cluster. A NodePort is published on every node, so the master address works too.

Run the verification playbook, which asserts rather than assumes:

```
$ ansible-playbook playbooks/verify.yml
```

It checks that both nodes are Ready and registered on the private network, that all eleven Deployments have their replicas available, that checkoutservice exists under the right name, that the

frontend Service is a NodePort, and that an HTTP request to the storefront returns 200 with recognisable content. Any failure stops the play.

Try the application

1. Browse the catalogue and open a product.
2. Add something to the cart, then reload the page — the cart persists because cartservice writes to redis-cart.
3. Complete a purchase. This is the path that exercises checkoutservice, paymentservice, shippingservice and emailservice together.

9. Part 5 — Teardown and Low-Memory Hosts

Removing things

```
$ ansible-playbook playbooks/teardown.yml   # app only, cluster stays
$ vagrant halt                              # stop the VMs, keep the disks
$ vagrant destroy -f                        # delete everything
```

10. Deliverables

Submit one archive named <student-id>.zip containing:

#	Item	Requirements
1	`repository/`	Your complete project as a git repository, including .git so the history is visible.
2	`report.pdf`	One section per part, answering every question posed, with the requested evidence.
3	`screenshots/`	Storefront, completed order, and any screenshot
4	`recap.txt`	PLAY RECAP output for: first successful run, the two idempotency runs

Security. Do not submit SSH private keys, kubeconfig files or cluster join tokens. A kubeconfig contains a client certificate that grants full administrative access to the cluster it came from. Any credential you submit will be treated as compromised.

Demo

You will run your project live in the lab. Expect to be asked to break something and fix it, or to explain any line in your own playbooks. A working submission that you cannot explain scores poorly.

12. Grading Rubric

No.	Area	What earns full marks	Marks
1	Working deployment	Clean run from destroyed VMs; all evidence present; storefront and checkout both demonstrated.	10

No.	Area	What earns full marks	Marks
2	Network diagnosis	Both experiments performed; symptoms described precisely and distinguished; iface-can-reach reasoning correct.	15
3	Bug diagnosis	Bug reproduced; incident report shows genuine investigation, not hindsight; prevention proposal is sound.	15
4	Ingress	Ingress controller installed via Ansible; hostname routing works; frontend returned to ClusterIP; verify.yml extended.	20
5	Probes and limits	Chart extended and vendored cleanly; limits measured not guessed; probes on four or more services; restart demonstrated.	20
6	Scaling	Scaling done through Ansible; drain performed and explained; PDB reasoning correct.	10
7	Idempotency	changed=0 on the second run; non-idempotent task identified or absence justified; judgement questions answered.	10
	Total		**100**

Appendix A — Service and Port Reference

Every Service in the microservice namespace. The frontend reaches its backends by these names; a rename anywhere breaks the caller silently.

Service	Port	Protocol	Called by
`frontend`	80	HTTP	Your browser, via NodePort
`productcatalogservice`	3550	gRPC	frontend, checkoutservice, recommendationservice
`currencyservice`	7000	gRPC	frontend, checkoutservice
`cartservice`	7070	gRPC	frontend, checkoutservice
`recommendationservice`	8080	gRPC	frontend
`shippingservice`	50051	gRPC	frontend, checkoutservice
`checkoutservice`	5050	gRPC	frontend
`adservice`	9555	gRPC	frontend
`paymentservice`	50051	gRPC	checkoutservice
`emailservice`	5000	gRPC	checkoutservice
`redis-cart`	6379	RESP	cartservice

Note on emailservice. Its container listens on 8080 but the Service publishes 5000, because that is what checkoutservice is configured to call. A Service port and a container port need not match — targetPort bridges them.

Appendix B — Troubleshooting

Symptom	Where to look
`vagrant up: address not allowed`	VirtualBox host-only range. See section 3.
`ansible k8s -m ping fails, permission denied`	The key paths in inventory.ini only exist after vagrant up completes.
kubeadm init fails on a preflight check	Almost always swap, or the VM has too little RAM. Check free -h on the VM.
Nodes stay NotReady after cluster.yml	CNI did not come up. kubectl -n kube-flannel get pods, then describe the failing pod.
Pods stuck in Pending	Nothing can schedule them. kubectl describe pod shows the reason — usually insufficient memory on node1.
Pods stuck in ContainerCreating	Usually image pull. kubectl describe pod, and check outbound access to gcr.io.
Storefront loads but products are missing	productcatalogservice. Check its pod logs and confirm the Service name resolves.
Checkout fails at the last step	checkoutservice naming. See section 7 — this is the deliberate upstream bug.
Cart empties between page loads	More than one redis-cart replica behind one Service, each with its own emptyDir.
`frontend Service stuck in <pending>`	It is a LoadBalancer. The NodePort override is missing.
`kubectl logs: connection refused`	The kubelet registered the NAT address. See section 6, trap 2.
Everything works on master, nothing on node1	Flannel is using the wrong interface. See section 6, trap 1.

Appendix C — Command Reference

Project

```
$ ansible-galaxy install -r requirements.yml
$ vagrant up | halt | destroy -f
$ ansible k8s -m ping
$ ansible-playbook site.yml
$ ansible-playbook playbooks/cluster.yml --tags kubernetes
$ ansible-playbook playbooks/verify.yml
```

On the control plane (vagrant ssh master, then sudo -i)

```
$ kubectl get nodes -o wide
```

```
$ kubectl get pods -A
$ kubectl -n microservice get pods,svc
$ kubectl -n microservice describe pod <name>
$ kubectl -n microservice logs <name> --tail=50
$ kubectl -n microservice exec -it <name> -- sh
$ helm -n microservice list
$ helm -n microservice get values frontendservice
```

Inspecting without applying

```
$ helm template frontendservice chart/microservice \
    -f values/frontend-values.yaml
$ ansible-playbook site.yml --check --diff
```

Before you submit. Run vagrant destroy -f and build the whole project again from nothing. A deployment that only works on the cluster you have been poking at by hand has not been tested, and asks for evidence from exactly that clean run.